

Função *recursiva*

Uma função **recursiva** chama a si mesma para resolver uma versão menor do mesmo problema, até chegar a um caso tão simples que a resposta é direta: o **caso base**. Nesta aula: a estrutura de toda recursão, fatorial com o rastro das chamadas, o que acontece sem caso base, o custo do Fibonacci ingênuo e onde a recursão realmente brilha.

Caso base

Caso recursivo

Pilha de chamadas

RecursionError

lru_cache

Recursão × laço

Estruturas aninhadas

Formação Claude Code 2026 · Baseado em docs.python.org (sys.getrecursionlimit, RecursionError, functools.lru_cache), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em [exemplos/](#) · Saídas reais

Uma função *que chama a si mesma*

contagem.py

```
def contagem_regressiva(n):  
    if n == 0:                # caso base: para aqui  
        print("Fogo!")  
        return  
    print(n)  
    contagem_regressiva(n - 1) # chama a si mesma, menor  
  
contagem_regressiva(3)
```

SAÍDA

```
3  
2  
1  
Fogo!
```

TODA RECURSÃO TEM DUAS PARTES

Caso base: quando parar, sem chamar de novo (`n == 0`).

Caso recursivo: chamar a si mesma com um problema **menor** (`n - 1`), que se aproxima do caso base.

Se o problema não diminui, ou se não há caso base, a função nunca para (seção 04).

O exemplo *clássico*

fatorial.py

```
def fatorial(n):  
    if n <= 1:                # caso base  
        return 1  
    return n * fatorial(n - 1)  # caso recursivo  
  
print(fatorial(5))    # 5 * 4 * 3 * 2 * 1  
print(fatorial(20))
```

SAÍDA

```
120  
2432902008176640000
```

- ✓ $5! = 5 \times 4!$ e $4! = 4 \times 3!$... a definição já é recursiva
- ✓ O caso base `n <= 1` devolve 1 e encerra a descida
- ✓ O `int` sem limite (aula 03) aguenta 20! sem estourar

O rastro *das chamadas*

rastro.py

```
def fatorial(n, nivel=0):
    recuo = "  " * nivel
    print(f"{recuo}fatorial({n})")
    if n <= 1:
        resultado = 1
    else:
        resultado = n * fatorial(n - 1, nivel + 1)
    print(f"{recuo}-> {resultado}")
    return resultado

fatorial(4)
```

SAÍDA

```
fatorial(4)
  fatorial(3)
    fatorial(2)
      fatorial(1)
        -> 1
      -> 2
    -> 6
  -> 24
```

- ✓ **Descida:** cada chamada espera a próxima (recuo aumenta)
- ✓ **Subida:** do caso base de volta, cada uma multiplica e devolve
- ✓ As chamadas em espera ficam numa **pilha** na memória

Sem caso base: *RecursionError*

sem_base.py

```
def infinita(n):  
    return infinita(n + 1)    # esqueceu o caso base  
  
infinita(1)
```

```
Traceback (most recent call last):  
  File "sem_base.py", line 4, in <module>  
    infinita(1)  
    ~~~~~^  
  File "sem_base.py", line 2, in infinita  
    return infinita(n + 1)    # esqueceu o caso base  
  File "sem_base.py", line 2, in infinita  
    return infinita(n + 1)    # esqueceu o caso base  
  File "sem_base.py", line 2, in infinita  
    return infinita(n + 1)    # esqueceu o caso base  
[Previous line repeated 996 more times]  
RecursionError: maximum recursion depth exceeded
```

- ✓ Sem caso base, a pilha cresce até o limite do Python
- ✓ O Python avisa "[Previous line repeated 996 more times]" e para com **RecursionError**
- ✓ O limite padrão é 1000 chamadas (seção 06)

Ao ver RecursionError, confira: existe caso base? A chamada recursiva chega mesmo nele (o problema está diminuindo)?

Fibonacci: *elegante, mas caro*

fibonacci.py

```
from functools import lru_cache

chamadas = 0

def fib(n):
    global chamadas
    chamadas += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(25), "com", chamadas, "chamadas")

@lru_cache(maxsize=None)    # guarda o que já calculou
def fib_rapido(n):
    return n if n < 2 else fib_rapido(n - 1) + fib_rapido(n - 2)

resultado = fib_rapido(25)
calculos = fib_rapido.cache_info().misses
print(resultado, "com", calculos, "cálculos")
```

SAÍDA

```
75025 com 242785 chamadas
75025 com 26 cálculos
```

- ✓ `fib(n)` chama `fib(n-1)` e `fib(n-2)`, que recalculam as mesmas coisas várias vezes
- ✓ Para `fib(25)`: **242.785** chamadas
- ✓ `@lru_cache` guarda cada resultado: só **26** cálculos

Dica do guia (cap. 15): `@functools.lru_cache` para funções puras. O `@` é um decorador, assunto mais avançado; aqui basta saber que ele "lembra" os resultados.

Recursão *ou* laço?

iterativo.py

```
def fatorial_loop(n):  
    resultado = 1  
    for i in range(2, n + 1):  
        resultado *= i  
    return resultado  
  
print(fatorial_loop(5), fatorial_loop(20))  
import sys  
print("Limite de recursão:", sys.getrecursionlimit())
```

SAÍDA

```
120 2432902008176640000  
Limite de recursão: 1000
```

	RECURSÃO	LAÇO (FOR/WHILE)
Leitura	Natural para problemas "de dentro para fora"	Natural para repetições simples
Memória	Uma entrada na pilha por chamada	Constante
Limite	~1000 chamadas por padrão	Nenhum
Em Python, para contar, somar ou repetir N vezes, o laço é o caminho normal. A recursão vale quando o próprio dado é aninhado (próxima seção).		

Estruturas *dentro de estruturas*

`pastas.py`

```
# Recursão brilha em estruturas que se repetem por dentro
projeto = {"src": {"app.py": None, "util": {"texto.py": None}},
          "README.md": None}

def listar(pasta, recuo=0):
    for nome, conteudo in pasta.items():
        print("  " * recuo + nome)
        if conteudo is not None:      # é uma subpasta
            listar(conteudo, recuo + 1)

listar(projeto)
```

SAÍDA

```
src
  app.py
  util
    texto.py
  README.md
```

- ✓ Uma pasta contém arquivos e **outras pastas**, que contêm...
- ✓ Não se sabe a profundidade: um laço fixo não resolve, a recursão sim
- ✓ Mesmo padrão: menus com submenus, comentários com respostas, JSON aninhado

O caso base aqui é implícito: um arquivo (`None`) não gera nova chamada.

Soma *dos dígitos*

Enunciado: escreva `soma_digitos(n)` recursiva. Caso base: número de um dígito. Caso recursivo: último dígito (`n % 10`) mais a soma do resto (`n // 10`).

Uma resposta possível

```
# Exercício: soma dos dígitos com recursão
def soma_digitos(n):
    if n < 10:
        return n
    return n % 10 + soma_digitos(n // 10)

print(soma_digitos(2026))    # 2 + 0 + 2 + 6
print(soma_digitos(7))
```

SAÍDA

```
10
7
```

- ✓ `//` e `%` da aula 06 fazem o problema diminuir
- ✓ $2026 \rightarrow 6 + \text{soma}(202) \rightarrow 2 + \text{soma}(20) \rightarrow 0 + \text{soma}(2) = 2$
- ✓ Desafio: inverta um texto com recursão

Teste o que *ficou*

1. O que é o caso base?

A) A primeira chamada

B) Quando a função para de chamar a si mesma

C) O maior valor

D) O return final

B.

2. Recursão sem caso base termina em:

A) Resultado 0

B) RecursionError

C) Laço infinito silencioso

D) None

B. Limite de ~1000 chamadas.

3. Por que o fib(25) ingênuo é lento?

A) Números grandes

B) Recalcula os mesmos valores muitas vezes

C) Usa float

D) Falta de return

B. Iru_cache resolve.

4. Onde a recursão faz mais sentido?

A) Somar uma lista

B) Percorrer pastas com subpastas

C) Repetir 10 vezes

D) Ler input

B. Dados aninhados.

O mesmo problema, *cada vez menor*

Toda função recursiva tem um **caso base** e um **caso recursivo** que se aproxima dele. As chamadas formam uma pilha; sem caso base, vem o **RecursionError**. `lru_cache` evita recálculos, laços resolvem repetições simples e a recursão brilha em dados aninhados.

Caso base

Problema menor

Pilha

RecursionError

`lru_cache`

Próxima aula → 20 · Parâmetros `*args` e `**kwargs`

Fontes: docs.python.org/3/library/sys.html#sys.getrecursionlimit; docs.python.org/3/library/exceptions.html#RecursionError; docs.python.org/3/library/functools.html#functools.lru_cache, consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.